

DES models systems that change state only at discrete instants (events). State is piecewise constant. The goal: **simulate stochastic behaviour** and **estimate statistics** (mean delay, queue length, etc.) with confidence bounds.

1 DES Core Theory

DES Taxonomy

A **Discrete Event System** has four defining properties:

- **Event-driven** — state changes only at event instants (not continuously)
- **Discrete state space** X — countable states (e.g. queue length $\in \{0, 1, 2, \dots\}$)
- **Stochastic** — event timing drawn from random distributions
- **Continuous time** — events can occur at any real-valued time $t \in \mathbb{R}$

Timed Automaton (TA) / Stochastic Timed Automaton (STA)

STA is a 6-tuple: $(E, X, \Gamma, p, p_0, G)$

- E — finite set of **events**
- X — finite set of **states**
- $\Gamma: X \rightarrow 2^E$ — **feasible events** function (which events can fire in state x)
- $p(x', \tau \mid x, e)$ — **transition probability**: joint distribution of next state x' and sojourn time τ
- $p_0(x)$ — **initial state** distribution
- $G(x, e)$ — **timer function**: distribution of time until event e fires from state x

State trajectory is **cadlag** (right-continuous, left-limit): $x(t) = \text{constant}$ between events; jumps at event instants; $x(t^-)$ = state just before jump.

Event List & Cassandras Simulation Algorithm

The simulation maintains an **event list** (priority queue ordered by scheduled fire time).

Cassandras Algorithm — 6 steps per event:

1. **Dequeue** the earliest event e^* from the event list
2. **Update time**: $t \leftarrow t_{e^*}$ (advance clock to fire time)
3. **Update state**: $x \leftarrow f(x, e^*)$ (apply state transition)
4. **Delete infeasible**: remove events $e \notin \Gamma(x)$ from the event list
5. **Add feasible**: for each new $e \in \Gamma(x)$ schedule fire time using $G(x, e)$
6. **Reorder**: sort event list by fire time

Inverse Function Method (ICDF)

To draw a random sample from distribution F , given $u \sim \text{Uniform}[0, 1]$:

$$T = F^{-1}(u)$$

Exponential distribution with rate λ (inter-arrival times, service times):

$$F(t) = 1 - e^{-\lambda t} \quad \Rightarrow \quad T = \frac{-\ln(1-u)}{\lambda}$$

In code: `u = rand(1); T = -log(1-u)/lambda;`

Markov Model Taxonomy

- **GSMP** (Generalised Semi-Markov Process) — multiple non-exponential timers active simultaneously; full STA
- **SMP** (Semi-Markov Process) — at most **one** non-exponential timer active at a time
- **Markov Process (CTMC)** — **all** timers exponential; memoryless; state alone determines future

For a CTMC: $\{x(n)\}$ is a DTMC (the sequence of visited states). For a GSMP: $\{(x(n), \tau_n)\}$ is the DTMC — you need the full timer vector τ_n to get the Markov property.

Composed State Spaces

For N independent subsystems with state spaces $X_1, X_2, \dots, X_N$:

$$X = X_1 \times X_2 \times \dots \times X_N$$

With a shared queue between nodes 1 and 2:

$$X = X_1 \times Q \times X_2$$

2 Output Analysis

Stochastic Simulation of CTMC/GSMP

Select $x(0) \in X$, then for $n = 1 : N - 1$ sample next state and timer:

$$P(x(n+1), \tau_{n+1} \mid x(n), \tau_n)$$

Produces a random **piecewise-constant realisation** $\{x(t), t \in [t_A, t_B]\}$.

Statistic of interest (fraction of time in set $A \subseteq X$):

$$\hat{P}_N(A) = \int_{t_A}^{t_B} I_{x(\tau) \in A} d\tau$$

Transient vs. Stationary Behaviour

A simulation started from $x(0)$ has an **initial transient** before reaching statistical stationarity. Measurements taken during the transient are **biased** — they depend on the starting state. After the transient the statistics stabilise (**stationary region**).

Restarted vs. Regenerative Simulation

Restarted

Run simulation M times, each of length N_{res}
Each restart is **independent** $\Rightarrow$ easy statistics
Good for **transient** statistics

Regenerative

Run once with long length $N_{\text{reg}} \gg N_{\text{res}}$
One long run; batches defined by **revisits to state $\bar{x}$**
Good for **long-term** / **stationary** statistics

Regeneration & Batch Statistics

For a DTMC $\{x(n)\}$, choose **regeneration state $\bar{x}$** (visited frequently). Let $J = \{i \mid x(i) = \bar{x}\}$ and define cycle intervals $I_k = \{J_k, \dots, J_{k+1} - 1\}$.

Per-cycle statistic: $s_k = S(x(J_k), \dots, x(J_{k+1} - 1))$

The sequence $\{s_k\}$ is **i.i.d.** — enabling simple mean/variance estimation.

Batch mean estimator (N events total, broken into batches B_n):

$$S_N = \frac{1}{N} \sum_{j=0}^{N-1} f(x_j) = \frac{1}{N} \sum_n B_n \quad \text{var}(S_N) = \frac{1}{N} \text{var}(B_n) \quad \hat{V}_B = \frac{1}{N} \sum_n (B_n - S_N)^2$$

- **Average state:** $f(x) = x$ or time-weighted: $f(T, x) = T \cdot x$
- **Frequency of occurrence:** $f(x) = I_{x \in A}$ time-fraction: $f(T, x) = T \cdot I_{x \in A}$

Confidence Interval

By the Central Limit Theorem (CLT) the sum S_N is approximately normal for large N .

$$95\% \text{ CI: } \hat{\mu} \pm 2 \hat{\sigma} \quad \text{where } \hat{\sigma} = \sqrt{\hat{V}_B / N}$$

2σ gives a **two-sided 95%** confidence interval (CLT; $\approx 13.6 + 34.1 + 34.1 + 13.6 = 95.4\%$). To tighten CI: increase N (more batches/restarts) since $\hat{\sigma} \propto 1/\sqrt{N}$.

Choosing the Regeneration State $\bar{x}$

Queue type	Regeneration state
M/M/1	Any queue length (memoryless both directions)
M/G/1	Any state after a departure (general service timer restarted)
G/M/1	Any state after an arrival (general arrival timer restarted)
G/G/1	State $\bar{x} = 1$ after arrival (both timers reset at that moment)

3 Exercise — CANsimple (TrueTime Regenerative Simulation)

System: CAN bus with two nodes. Node 1 generates packets with exponential inter-arrival times (rate λ). A high-priority node injects competing traffic. Node 2 receives and measures end-to-end delay.

Goal: Estimate mean packet delay using **regenerative simulation** with confidence bounds.

Simulink / TrueTime File Structure

File	Role
CANsimple.slx	Simulink model (TrueTime kernel, CAN network, scope)
generator_code.m	Node 1 task: generates packets, implements inverse-CDF arrivals
highprio_code.m	High-priority node task: competing Poisson traffic
send_queue_task.m	Sender task: dequeues and transmits packets periodically
receiver_code.m	Node 2 task: measures delay, detects regeneration state
regen_analysis.m	Post-simulation script: computes mean, std, 95% CI

Key Code Patterns

generator_code.m — Poisson arrivals via inverse function method:

```
u = rand();
T = -log(1-u) / data.lambda; % exponential inter-arrival (ICDF)
ttCreateJob('generator_task', ttCurrentTime + T);
Each call also stamps msg.ttime = ttCurrentTime and records msg.cycle = currentCycle.
```

receiver_code.m — measures delay and detects regeneration:

```
delay = ttCurrentTime - msg(1).ttime; % end-to-end delay
regen.sum(c) = regen.sum(c) + delay;
regen.count(c) = regen.count(c) + 1;
```

```
% Regeneration: queue empty AND no packet in flight
if isempty(msgQueue) && origInFlight == 0
    regen.mean(end+1) = regen.sum(c) / regen.count(c);
    currentCycle = currentCycle + 1;
end
```

Regeneration state $\bar{x}$: queue empty **and** no packet in transit. This is equivalent to M/G/1 “after departure” state — all timers reset.

regen_analysis.m — post-simulation statistics:

```
cycleMeans = regen.mean; % mean delay per cycle
warmupCycles = 5; % discard initial transient
cycleMeansUsed = cycleMeans(warmupCycles+1:end);

meanDelay = mean(cycleMeansUsed);
stdCycles = std(cycleMeansUsed);
stdEstimate = stdCycles / sqrt(n); % std of mean estimate
ciLow = meanDelay - 2*stdEstimate; % 95% CI lower bound
ciHigh = meanDelay + 2*stdEstimate; % 95% CI upper bound
```

What origInFlight Does

A regeneration cycle can only close when the system is truly empty:

- msgQueue empty — no packets waiting to be sent
- origInFlight == 0 — no packet submitted to CAN but not yet received

Both conditions needed; otherwise a cycle would close while a packet is still travelling on the bus, giving an incorrect (incomplete) cycle statistic.

Restarted vs. Regenerative in This Exercise

Restarted (not used here)	Regenerative (used here)
sim('CANsimple', N) called M times	sim('CANsimple', 200) called once
Each run fully independent	Batches split by $Q = 0$ & in-flight = 0 revisits
SimData{i}(end,1) = final counter	regen.mean(k) = mean delay of cycle k
Variance: $(\text{SimData}\{i\}(\text{end},1) - m)^2/M$	Variance: $\text{std}(\text{cycleMeans})^2/n$

Summary: Full Simulation Workflow

1. TrueTime kernel starts; generator_code fires at $t = 0$, schedules next job via inverse-CDF.
2. Each generated packet is pushed onto msgQueue with timestamp and cycle number.
3. send_queue_task dequeues one packet per period and calls ttSendMsg; increments origInFlight.
4. High-priority node also injects competing packets; these do **not** carry cycle info.
5. receiver_code receives each packet; for source 1 packets: computes delay, accumulates cycle sum.
6. When msgQueue empty **and** origInFlight = 0: close cycle, record regen.mean, increment currentCycle.
7. After simulation: regen_analysis.m discards warm-up cycles, computes mean $\pm 2\sigma$ CI.